

CogAge Atomic Behaviour: optimizer comparison

Objective

Compare eight update configurations for four-class activity classification using a fixed fully connected network. The held-out test set is not used to choose epochs or optimizer settings.

Selected classes and labels

Bending=0, Walking=1, Sitting=2, Standing=3. The four labels come from the activity token in the folder name, not from a label column in the raw sensor files.

Original split and window counts

For every selected class, `_1_extracted` is the training recording and `_2_extracted` is the test recording. The raw recordings provide 10 Events per split and class. Windowing yields 10 complete windows per class in original train and 10 per class in held-out test. From original training only, 2 Events per class (8 windows total) are assigned to validation; 8 per class (32 total) remain for model fitting. Test remains 40 windows.

Input preparation

Semicolon-delimited sensor files use epoch-millisecond timestamps. The selected folders contain 12 sensor types with differing measurement widths and sampling intervals. Streams are averaged onto a 20 ms grid, linearly interpolated, synchronized within each Event, and divided into non-overlapping 128-step windows. Each window has 33 features. Raw audit: 960 files, zero malformed rows and zero missing raw measurement values. Resampling introduced sparse-grid NaNs that were interpolated; incomplete event tails were discarded.

Validation and limitations

The validation split groups windows by original folder and Event and is seeded. All recordings are from Subject 1, so this cannot measure generalization to unseen participants. The final model-training set contains only 32 windows, which is very small relative to the network capacity.

Network and experimental configurations

Input: 128 x 33 flattened to 4224; architecture: 4224 -> 32 -> 16 -> 4 logits; ReLU hidden activations; 135,796 trainable parameters.

Weights use the same He-normal initialization seed (42) in each run. Loss is sparse categorical cross-entropy from logits, computed with numerically stable softmax. No softmax is added to the model output.

Configuration	Batch size	Updates / epoch
Full-batch GD	32	1
Mini-batch GD	8	4
Stochastic GD	1	32
SGD + Momentum	8	4
Adagrad	8	4
RMSprop	8	4
Adam	8	4
AdamW	8	4

Fixed settings

100 epochs; learning rate 0.01 for every method; mini-batch size 8; momentum 0.9; RMSprop decay 0.99; Adam/AdamW betas (0.9, 0.999); AdamW decoupled weight decay 0.01. All experiments use the same data, initialization, loss, and maximum epochs. Plain NumPy optimizer updates are implemented directly.

Runtime measures training plus per-epoch train/validation scoring; the one-time held-out test evaluation is excluded. Epochs are not directly comparable as update counts differ substantially.

Reproducibility: seeds 42/43 for model initialization and batch ordering. Run the preprocessing script, then execute this notebook top to bottom. Each experiment writes to a fresh timestamped subfolder.

Training and validation loss

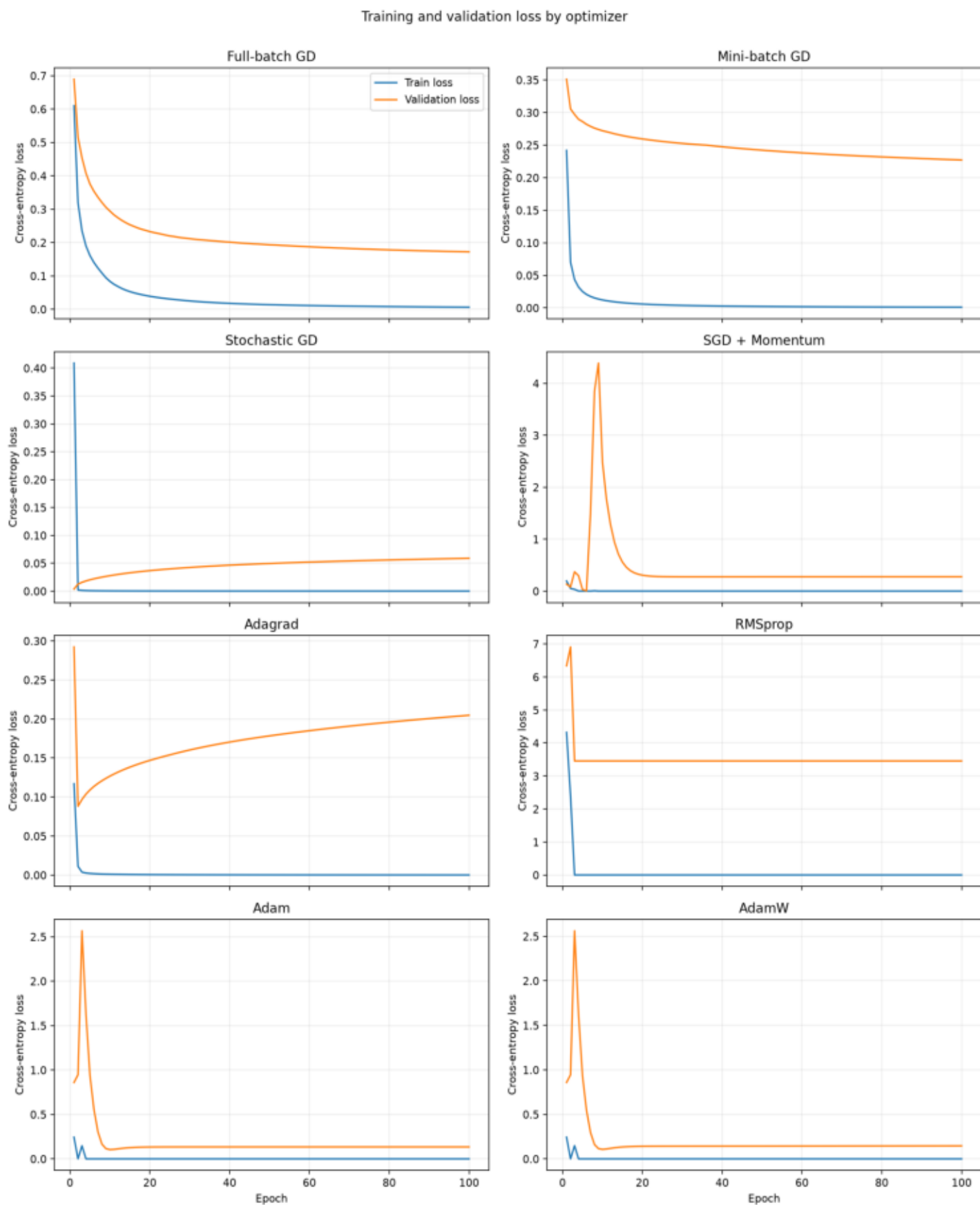


Figure 1. Cross-entropy loss by epoch for all eight optimizers. Each panel uses its own y scale so method-specific changes remain visible. Checkpoints are selected using validation loss only.

Training and validation accuracy

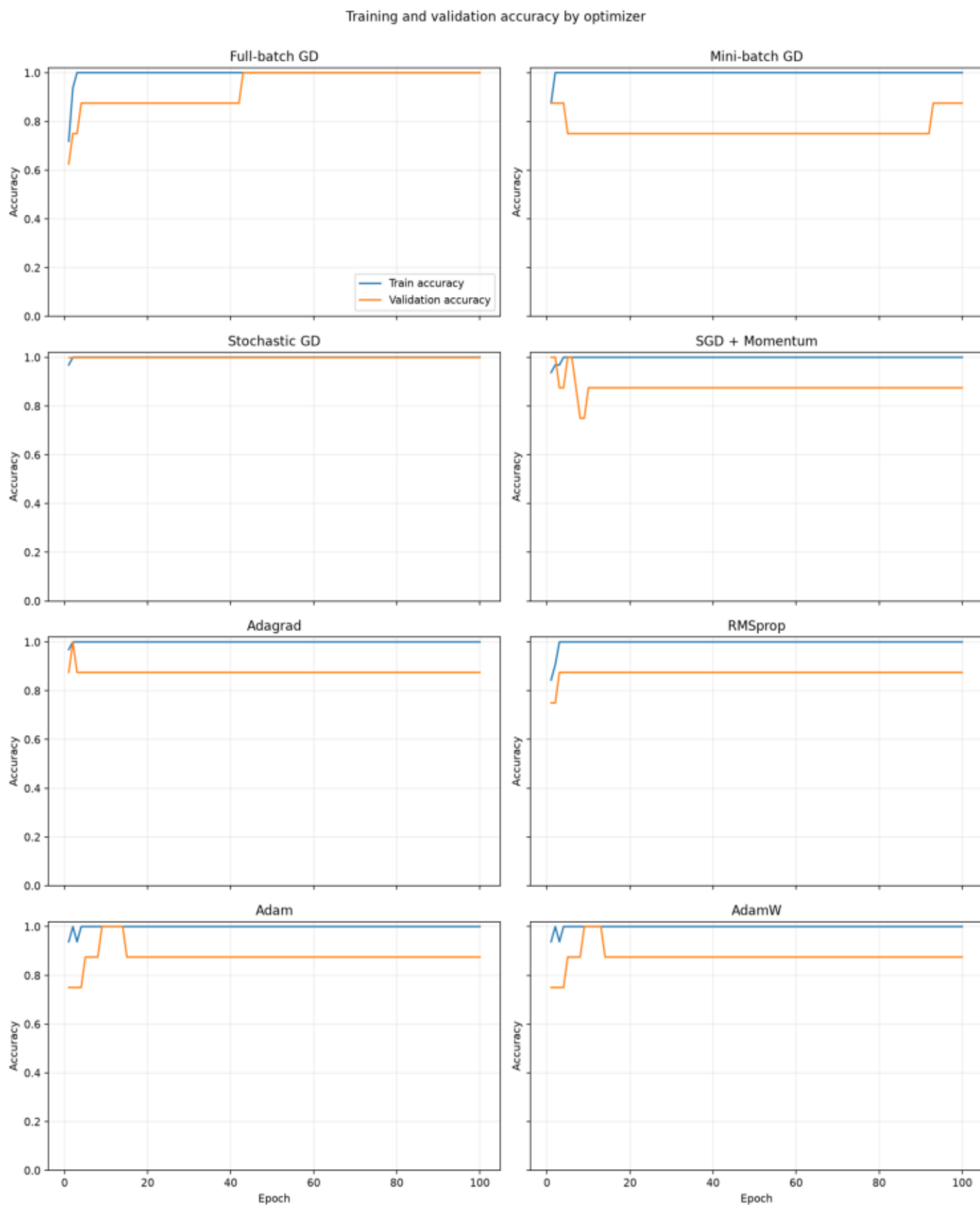


Figure 2. Training and validation accuracy by epoch. Validation contains only eight windows (two per class), so each error changes validation accuracy by 12.5 percentage points.

Convergence and held-out test metrics

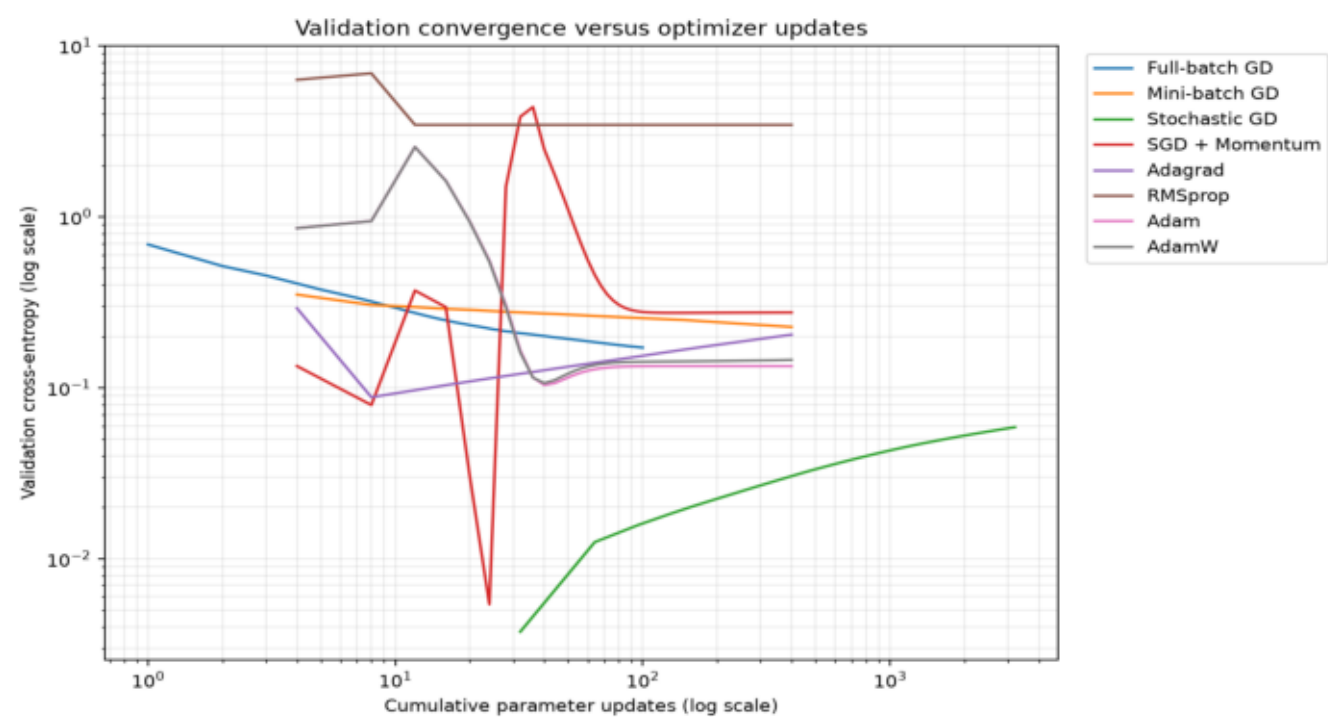


Figure 3. Validation loss versus cumulative parameter updates (both axes logarithmic). Full-batch GD makes one update per epoch; mini-batch configurations make four; stochastic GD makes 32.

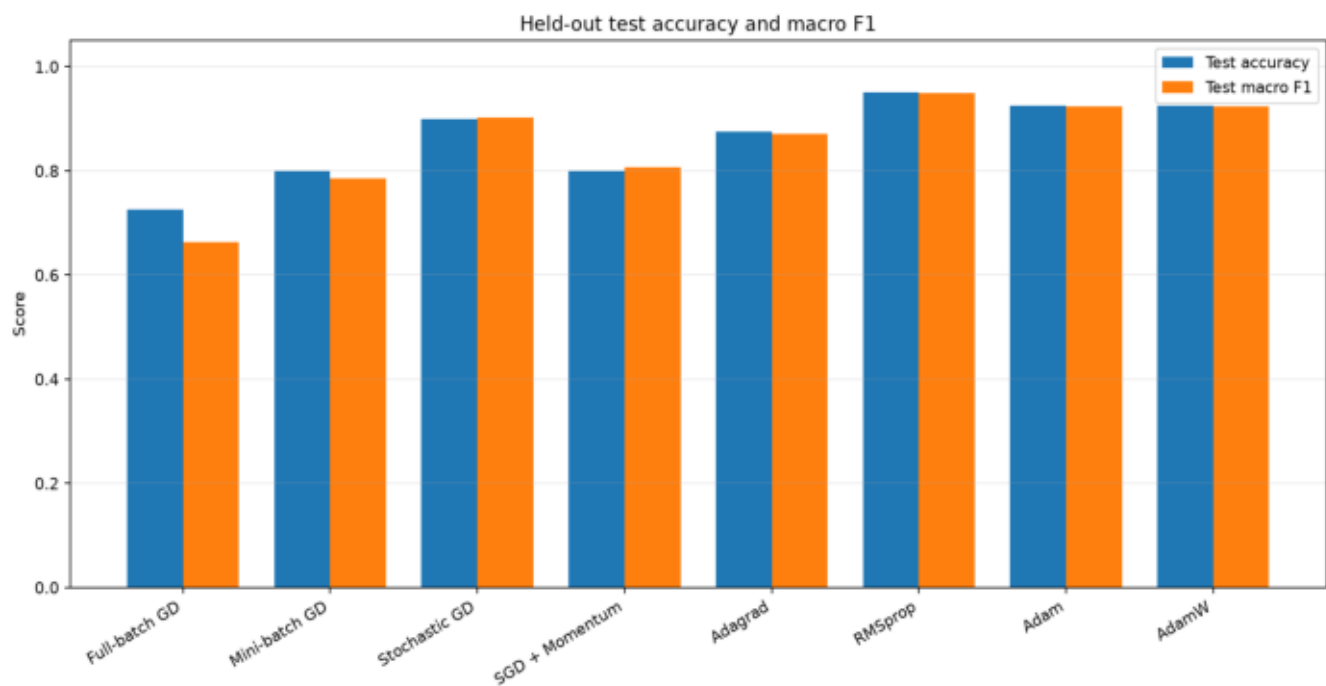


Figure 4. Test accuracy and macro F1 for the checkpoint chosen by validation loss. Test data is evaluated once per run and is not used for model selection.

Test results, confusion matrices, and interpretation

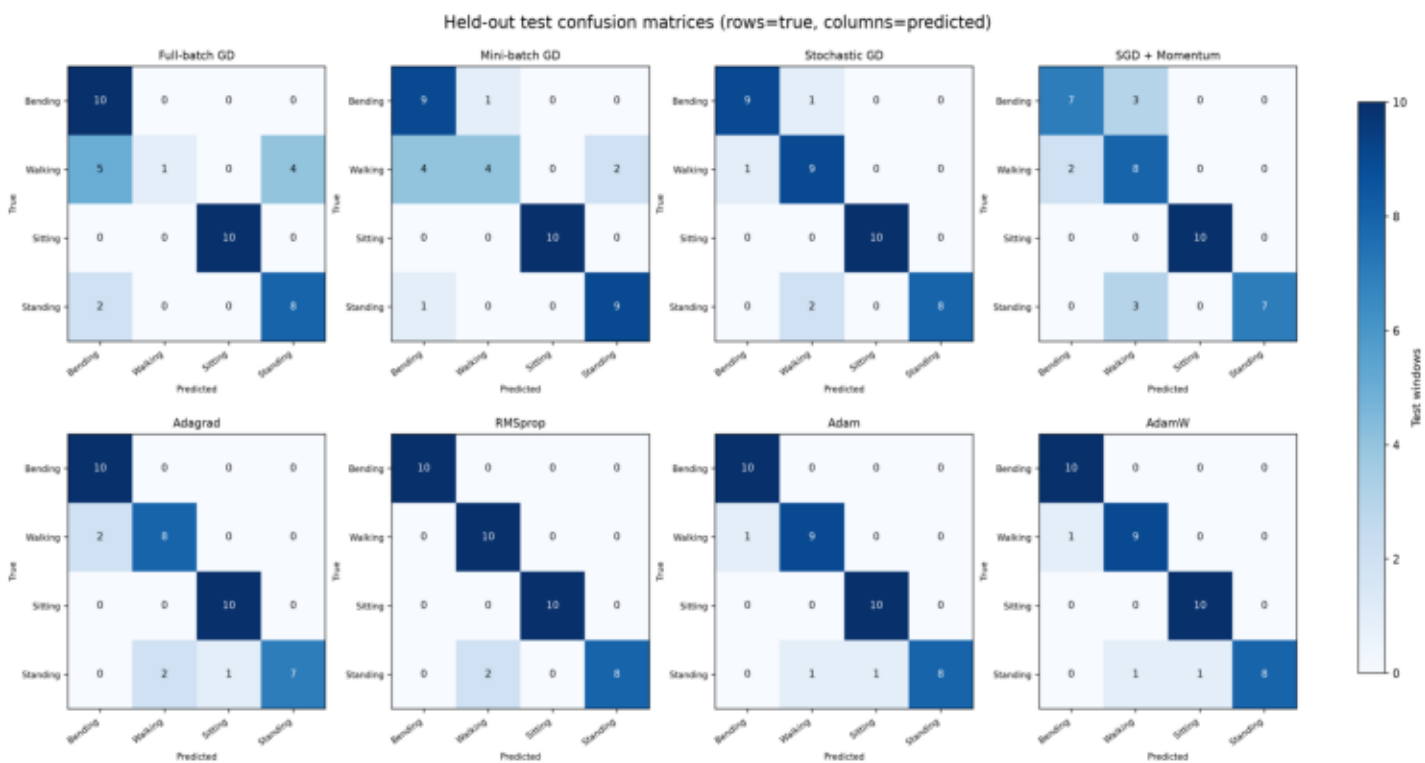


Figure 5. Per-optimizer 4 x 4 confusion matrices. Rows are true classes; columns are predicted classes.

Optimizer	Best val loss	Epoch	Test acc.	Macro F1	Seconds	Updates
Full-batch GD	0.172	100	0.725	0.662	0.54	100
Mini-batch GD	0.227	100	0.800	0.785	0.88	400
Stochastic GD	0.004	1	0.900	0.902	4.91	3,200
SGD + Momentum	0.005	6	0.800	0.807	1.46	400
Adagrad	0.088	2	0.875	0.871	2.10	400
RMSprop	3.454	3	0.950	0.949	2.34	400
Adam	0.104	10	0.925	0.923	2.77	400
AdamW	0.107	10	0.925	0.923	2.94	400

In this run, RMSprop had the highest test accuracy (95.0%; macro F1 0.949). RMSprop also had a high test score but a very poor best validation loss, showing the instability of selecting on only eight validation examples. Stochastic GD reached strong validation/test scores but required 3,200 updates and the longest measured time. Full-batch GD used the fewest updates and least time, but its test score was lower. Adam and AdamW were nearly identical here; this small run does not establish a general optimizer ranking.

Limitations: one subject, only 32 model-training and 8 validation windows, and 135,796 parameters. The validation estimates are noisy, and results are not participant-independent. No class balancing was needed because each class had equal window counts. Do not interpret the held-out test results as evidence of broad population generalization.